Hover v0.7.0 Language Manual

A Mixed-Systems Co-Simulation Language

Eray Mercan

Contents

1 Overview **1**

2 File Structure & Imports **1**
2.1 The `import` Directive . . . 1

3 Simulation Directives **1**
3.1 .tran — Transient Analysis . . . 1
3.2 .save — Signal Logging . . . 2
3.3 .zcd — Zero-Crossing Detection . . . 2
3.4 .op — DC Operating Point . . . 2
3.5 .solver — Solver Type . . . 3

4 Data Types **6**
4.1 Structural Types . . . 6
4.2 Floating-Point Types . . . 6
4.3 Integer Types . . . 6
4.4 Arrays . . . 6
4.4.1 Multidimensional Arrays . . . 7
4.5 Pointers . . . 7
4.5.1 Address-of and Dereference . . . 7
4.5.2 Indexing . . . 8
4.5.3 Pointers as Function Parameters . . . 8
4.5.4 Pointers as State . . . 8

5 Number Literals & Suffixes **8**

6 Operators **9**
6.1 Arithmetic . . . 9
6.2 Bitwise & Integer Operators . . . 9
6.3 Comparison (return 1.0 or 0.0) . . . 9
6.4 Logical . . . 10
6.5 Precedence (highest to lowest) . . . 10

7 Variables & State **10**
7.1 Local Variable . . . 10
7.2 State Variable . . . 10

8 Control Flow **10**
8.1 If / Else If / Else . . . 10
8.2 While Loop . . . 11

9 Functions **11**
9.1 Declaration . . . 11
9.2 Call . . . 12

10 Modules **12**
10.1 Domains . . . 12
10.2 Domain Restrictions . . . 13
10.3 Why Two Computational Domains . . . 13
10.4 Bracket Convention . . . 14
10.5 Declaration . . . 14
10.6 Instantiation . . . 15
10.7 Execution Order . . . 15

11 C/C++ Integration (FFI) **16**
11.1 The `importc` Directive . . . 16
11.2 `extern func` Declarations . . . 16
11.3 Putting it Together . . . 16
11.4 Handling Structs and Complex Types . . . 17
11.5 The Opaque Pointer Pattern . . . 17

12 Physical Primitives **18**
12.1 Driven Sources (Logic-controlled) . . . 18
12.2 Fixed Sources (Compile-time value) . . . 19

13 Built-in Intrinsics **19**
13.1 nr_prev() . . . 19
13.1.1 nr_prev() — Supported Forms . . . 20

14 Special Keywords **21**

15 Simulation Run Order **21**

16 Full Example **22**

1. Overview

Hover is a co-simulation language that combines **logic** (software-style computation) with **physical** (circuit) descriptions in a single source file. The compiler flattens a module hierarchy into a netlist, solves the circuit using Modified Nodal Analysis (MNA) at each timestep, and executes the logic blocks between steps.

2. File Structure & Imports

Hover supports modularizing your code across multiple source files. You can bring external components, such as device models from the standard library or custom sub-circuits, into your current file using the `import` directive.

2.1 The `import` Directive

Imports must be declared at the top of the file, outside of any module or function definitions. The target file is specified as a string literal, representing a path relative to the current file's directory or the compiler's include path.

```
// Import standard math functions from the standard library
import </math/math.hvr>;

// Import a standard BJT device model from exact path
import "./standard_library/semiconductors/npn_bjt.hvr";

module main<>()[] {
    // NPN from the imported file is now available in the global
       scope
    module q1 = NPN<1e-14, 100, 0.026>()[base, coll, emit];
}
```

3. Simulation Directives

Directives appear at the top level, outside any module, and start with .

3.1 .tran — Transient Analysis

```
.tran(start, stop, step/min_step, max_step);
```

Argument	Description
`start`	Start time (currently always 0)
`stop`	End time
`step/min_step`	Timestep on fixed solvers, minimum step size on adaptive solvers (*dt*)
`max_step`	Max step size on adaptive solvers, not compulsory on fixed solvers

```
.tran(0, 40m, 1u); // 0 to 40 ms, 1 us steps -> 40 000 points
```

3.2 .save — Signal Logging

```
.save(signal1, signal2, ...);
```

Signals are referenced by their fully-qualified mangled name using dot notation.

```
.save(main.lc_out, main.generator.carrierWave);
```

- If the name matches an MNA node it is read from the circuit solution vector.
- Otherwise it is read from the logic values map (VM signal).

Results are exported to `simulation_output.csv`.

3.3 .zcd — Zero-Crossing Detection

```
.zcd;
```

Enables zero-crossing detection for circuits whose logic outputs switch discretely — comparators, PWM generators, digital controllers. When active, the solver probes one step ahead at each timestep before committing. If any logic variable jumps by more than 1.0 between the current state and the probe, a discontinuity is declared and the crossing time is isolated by bisection down to 1 ps resolution. The timestep is then shortened to land exactly on the event boundary, preventing the integrator from stepping across a discontinuity and introducing phase error or false transients.

Bisection tolerance: 10^{-12} s (1 picosecond).

Performance: the detection check is entirely skipped when `.zcd` is absent — analog-only circuits pay no overhead.

When to use: any circuit where a logic block produces a step output, such as a PWM comparator, a relay model, or a digital PI controller whose output saturates.

3.4 .op — DC Operating Point

```
.op;
```

Instructs the VM to solve for the DC operating point before the transient begins. Without `.op`, all capacitors start uncharged and voltage sources snap to their values in a single linear solve — the circuit charges from zero initial conditions, producing a potentially long startup transient before reaching steady state.

With `.op`, the solver uses source stepping: the supply voltages and driven sources are scaled from 5 % to 100 % in twelve alpha steps. At each alpha level, up to 500 Newton-Raphson iterations are run with a 0.05 V per-iteration damping limit. The converged solution at $\alpha = 1.0$ is committed as the initial condition for the transient, so the waveform begins at the settled operating point.

When to use: any circuit with a well-defined DC bias point — amplifiers, filters, motor drives. Omit `.op` only when the startup transient itself is the subject of the simulation.

3.5 .solver — Solver Type

```
.solver(solverName, reltol, abstol, max_iter);
```

Same with SPICE.

Solvers

Implicit solvers treat the circuit as a Differential-Algebraic system and solve $Gx = b$ at each timepoint, enforcing Kirchhoff's laws and voltage source constraints simultaneously. They are unconditionally stable for linear circuits. Nonlinear devices require an inner Newton-Raphson loop. Every HVR implicit solver uses **Modified Newton**: the Jacobian is reused across multiple inner iterations rather than recomputed every iteration, reducing cost relative to a full Newton scheme. Reuse policy differs by solver — some recompute once per timestep unconditionally, others carry a Jacobian across many timesteps and only refresh it once a convergence failure proves it stale — see each solver's entry below.

Per-iteration step damping also differs by solver. All use **diagonally-scaled trust-region** damping: each variable's proposed step is measured relative to its own current magnitude rather than against a single global voltage budget, so the same solver behaves correctly on a millivolt-scale junction and a kilovolt-scale bus within one circuit.

Solver	**Description**	**Simulink**	**SPICE**
`euler_fixed`	Backward Euler, fixed timestep, first-order, no tunable parameters. A single implicit solve per step with no error control. Use as a debugging baseline or when a known-simple, always-converges reference trace is needed.	`ode1`	`N/A`
`euler_adaptive`	Backward Euler with Modified Newton-Raphson, diagonally-scaled trust-region damping, and convergence-driven adaptive timestepping. Expands dt by 1.5× when the inner loop converges quickly, contracts by 0.5× on failure. General-purpose first-order solver when a fixed-order, fixed-step solver is too slow.	`ode15s` (1st order)	`N/A`
`gauss_siedel`	Backward Euler with fixed-point relaxation under-relaxation ($\omega = 0.05$). No Jacobian formed at all — structurally independent of every Jacobian-related failure mode the other solvers share. Converges slowly but predictably; useful as a diagnostic cross-check when a Newton-based solver's convergence behavior is in question, since a difference in outcome isolates the problem to the Jacobian/Newton machinery rather than the circuit itself. Not recommended for general use.	N/A	N/A
`trapezoidal`	Variable-step Crank-Nicolson with Modified Newton-Raphson and diagonally-scaled trust-region damping. A-stable. Falls back to Backward Euler for one step after a ZCD event or a rejected step, to suppress numerical ringing before resuming 2nd-order accuracy.	`ode23t`	`Trap`

Solver	**Description**	**Simulink**	**SPICE**
`trapezoidal_fixed`	The fixed-step counterpart to `trapezoidal`: same 2nd-order Crank-Nicolson formula and the same Modified Newton loop, but with diagonally-scaled trust-region damping and no adaptive step-size control at all. Intended as a foundation-testing tool — isolates whether a convergence failure comes from the core Newton/Jacobian mechanics or from the adaptive step-size layer on top of it.	`ode23t`	`Trap`
`bdf2`	2nd-order Backward Differentiation Formula (Gear's method) with Modified Newton-Raphson, diagonally-scaled trust-region damping, and adaptive timestepping. L-stable — actively damps spurious oscillations after switching events. Uses a Backward Euler primer on startup and after any step rejection.	`ode15s`	`Gear`
`ndf2`	Shampine & Reichelt's order-2 Numerical Differentiation Formula — the real algorithm underlying `ode15s` — with diagonally-scaled trust-region damping. Adds a κ-weighted correction term to plain BDF2, improving accuracy within the same A-stable order-2 regime (per the Dahlquist second barrier, no linear multistep method of order 3 or higher can be A-stable, so this solver deliberately does not extend past order 2). Starts at order 1 and promotes to order 2 once enough step history exists.	`ode15s`	`Gear` (NDF variant)

4. Data Types

Hover provides distinct data types to represent both physical electrical nets and software-style logic variables.

4.1 Structural Types

Structural types are used purely for circuit topology and cannot hold logic states or numerical values.

Type	Description	Example
`wire`	Electrical net (topological only)	`wire in, out;`

4.2 Floating-Point Types

Floating-point numbers represent the continuous domains of time, voltage, and current.

Type	Description	Example
`double`	64-bit floating point	`double x = 3.14;`
`float`	32-bit floating point	`float y = 1.0;`

Backend Execution Note: While `float` variables are executed as true 32-bit C++ floats locally inside `digital` module logic, the underlying MNA solver and global state maps operate exclusively on 64-bit precision. Therefore, `float` values are implicitly promoted to `double` when crossing into analog modules, driving physical sources, or being logged via `.save`. For all physics and solver-related calculations, `double` is highly recommended to prevent numerical instability during Newton-Raphson iterations.

4.3 Integer Types

Integer types are primarily used for discrete digital logic, state machines, loop counters, and bitwise operations.

Type	Description	Example
`int`	Signed 32-bit integer	`int n = -4;`
`unsigned int`	Unsigned 32-bit integer	`unsigned int k = 42;`

Assigning a floating-point value to an integer variable implicitly truncates the fractional portion. Furthermore, the `unsigned` keyword is strictly reserved for integer types; an `unsigned double` is invalid.

4.4 Arrays

Append `[size]` to any type declaration to create fixed-size arrays:

```
wire[8]   databus;
double[3] coeffs = {1.0, 2.5, 0.5};
int[4]    counts = 0;    // all elements initialised to 0
float[4]  gains  = 31u; // all elements initialised to 31e-6
```

Note: `wire` arrays cannot be initialised with a value literal, as they do not hold data.

4.4.1 Multidimensional Arrays

Append additional `[size]` suffixes to declare arrays of two or more dimensions. Dimensions are read outer-to-inner, matching C:

```
double[2][3] m      = {{1.0, 2.0, 3.0}, {4.0, 5.0, 6.0}};
int[2][2]    grid   = 0;     // every element initialised to 0
float[3][3]  kernel = 5u;    // every element initialised to 5e-6
```

Initialisation follows the same two forms as a single-dimension array, applied recursively at every nesting level:

- **Nested brace literals** — `{{...}, {...}}` — assign each inner list to the corresponding row. The nesting depth must match the number of declared dimensions.
- **Scalar-fill** — a single value broadcasts to *every* element across all dimensions, not just the outermost one. `int[2][2] grid = 0;` zero-fills all four elements, exactly like the one-dimensional case.

Indexing a multidimensional array peels off one dimension per subscript, left to right:

```
double[2][3] m    = {{1.0, 2.0, 3.0}, {4.0, 5.0, 6.0}};
double[3]    row0 = m[0];    // one subscript -- the first row
double       x    = m[1][2]; // two subscripts -- single element,
    value 6.0
```

When a multidimensional array is passed as a function argument, only its outermost dimension is allowed to decay; every inner dimension must remain a fixed, known size:

```
func double sumRow(double[3] row) {
  return row[0] + row[1] + row[2];
}
```

Note: as with one-dimensional arrays, `wire` arrays of any dimensionality cannot be initialised with a value literal.

4.5 Pointers

Append `*` to any numeric type declaration to create a pointer. Pointer stars stack the same way C's do — one `*` per level of indirection — and follow the same declaration grammar as arrays, so the two can be combined:

```
double*  p;       // pointer to a double
int**    pp;      // pointer to a pointer to an int
double*[4] parr;  // fixed-size array of 4 double pointers
```

A pointer declared without an initializer defaults to `null`, exactly like a scalar defaults to `0`.

4.5.1 Address-of and Dereference

`&` takes the address of a variable, producing a pointer to its type; `*` in prefix position dereferences a pointer, producing its pointee's type. These are the same two symbols used elsewhere as bitwise AND and multiplication — Hover disambiguates purely by position, exactly as C does: in front of an operand with nothing to its left, they are unary; between two operands, they are binary.

```
double x = 3.14;
double* p = &x;   // p now holds the address of x
double  y = *p;   // y == 3.14, read through the pointer
```

```
4
5 *p = 2.5;          // write through the pointer -- x is now 2.5
```

4.5.2 Indexing

A pointer can be subscripted with `[]` exactly like an array — `p[i]` is equivalent to `*(p + i)` in the underlying generated code, following normal C pointer/array duality:

```
1 double* p = &coeffs[0]; // coeffs from the Arrays example above
2 double  first = p[0];   // same as coeffs[0]
```

4.5.3 Pointers as Function Parameters

Unlike scalar parameters (which are copied), a pointer parameter is aliased: the function receives the same address the caller passed, so writes made through it inside the function are visible to the caller after it returns. This mirrors how array parameters behave (§4.4).

```
1 func void increment(double* p) {
2   *p = *p + 1;  // modifies the caller's variable directly
3 }
4
5 double x = 5.0;
6 increment(&x); // x is now 6.0
```

4.5.4 Pointers as State

Because a pointer is just an address-sized value, it can be declared `state` to persist across simulation steps – the standard way to hold onto a handle returned by external C/C++ code for the lifetime of the simulation:

```
1 state int* handle = create_motor(); // see Section 10.5, The Opaque
    Pointer Pattern
```

Note: arithmetic on pointers (e.g. `p + 1`) is not currently supported – use indexing (`p[i]`) to walk through memory instead.

5. Number Literals & Suffixes

Engineering suffixes are part of the number token — no space between digits and suffix. Also scientific e notation is supported (1e-6, 1e3).

Suffix	Multiplier	Example	Value
`p`	10^{-12}	`10p`	10 pF / ps
`n`	10^{-9}	`100n`	100 nH / ns
`u`	10^{-6}	`2u`	2 μF / μs
`m`	10^{-3}	`2m`	2 mH / ms
(none)	1	`400`	400
`k`	10^{3}	`20k`	20 000
`Meg`	10^{6}	`1Meg`	1 000 000
`G`	10^{9}	`2G`	2 000 000 000

```
R<1k>()[a, b];     // 1 000 Ohm
C<2u>()[node, gnd]; // 2e-6 F
L<2m>()[in, out]; // 2e-3 H
```

6. Operators

6.1 Arithmetic

Operator	Description
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`**`	Power
`%`	Modulo

6.2 Bitwise & Integer Operators

Bitwise operators manipulate the underlying binary representations of data.

Operator	Description
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`«`	Left shift
`»`	Right shift
`~`	Bitwise NOT (unary)

Important: Bitwise operators are strictly restricted to `int` and `unsigned int` types. Applying them to `double` or `float` variables will trigger a semantic error at compile time.

Note: `&` and `*` are overloaded by position. Between two operands they mean bitwise AND and multiplication respectively; in front of a single operand with nothing to their left, they instead mean address-of and dereference – see §4.5, Pointers.

6.3 Comparison (return 1.0 or 0.0)

Operator	Description
`==`	Equal
`!=`	Not equal
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal
`<=`	Less than or equal

6.4 Logical

Operator	Description
`&&`	Logical AND
`||`	Logical OR
`!`	Logical NOT

6.5 Precedence (highest to lowest)

Operators	Description
`f()`, `a[]`	Function call, array subscript
`-x`, `!x`, `~x`	Unary minus, logical NOT, bitwise NOT
`&x`, `*x`	Address-of, dereference (unary)
`**`	Power
`*`, `/`, `%`	Multiplicative
`+`, `-`	Additive
`«`, `»`	Bitwise shift
`>`, `<`, `>=`, `<=`	Relational
`==`, `!=`	Equality
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`&&`	Logical AND
`||`	Logical OR

7. Variables & State

7.1 Local Variable

Re-initialised to the given value every simulation step.

```
double x = 0;
double a = 1.5, b = 2.5; // multiple declarations
```

7.2 State Variable

Initialised once at $t = 0$, then retains its value across timesteps. Equivalent to a register or flip-flop in hardware.

```
state double integrator = 0;  // starts at 0, keeps value each step
state double prev_error  = 0;
```

8. Control Flow

8.1 If / Else If / Else

```
if (condition) {
    // ...
} else if (other_condition) {
    // ...
} else {
    // ...
}

if (ref > carrier) {
    output = vdc;
} else {
    output = -vdc;
}
```

8.2 While Loop

```
while (condition) {
    // ...
}

// Modulo via repeated subtraction
double T = 1.0 / freq;
double t = time;
while (t > T) {
    t = t - T;
}
```

9. Functions

Functions are called at runtime (not flattened). They can only accept and return logic types (`double`, `int`, `float`). Wires and physical primitives are not allowed as parameters.

9.1 Declaration

```
func <return_type> <name>(<type> <param>, ...) {
    // body
    return expression;
}

func double sineWave(double freq) {
    double pi = 3.14159265;
    double w  = 2.0 * pi * freq;
    return sin(w * time);
}

func double triangleWave(double freq) {
    double T = 1.0 / freq;
    double t = time;
    while (t > T) {
        t = t - T;
    }
    double result = 0;
    if (t < T / 2.0) {
        result = (4.0 * t / T) - 1.0;
```

```
    } else {
        result = 3.0 - (4.0 * t / T);
    }
    return result;
}
```

9.2 Call

```
double ref     = sineWave(50);
double carrier = triangleWave(20k);
```

10. Modules

Modules are the primary building block of a Hover program. They are *flattened* at compile time — there is no runtime module overhead, only the resulting nets and logic objects. Every module belongs to exactly one domain, declared by a prefix keyword. The domain controls what the module may contain and when it executes each timestep.

10.1 Domains

Domain	Allowed	Forbidden
`analog module`	Physical primitives, `V()`/`I()`, math, `if`, function calls	`state`, `while`
`digital module`	`state`, `if`, `while`, math, function calls	Physical primitives
`module`	Wire declarations, `V()`/`I()` reads, `state`, function calls, sub-module instantiation	Computation, `if`, `while`

Analog modules model continuous-time device physics. State lives in physical elements (capacitors, inductors) managed by the MNA engine — not in software variables. `if` is permitted for clamping and saturation detection; `while` is forbidden because it risks infinite loops inside the Newton-Raphson convergence loop. This makes `analog module` the mechanism for defining custom device models — transistors, diodes, transformers — that stamp directly into the MNA matrix alongside built-in primitives.

Digital modules model discrete-time and event-driven behaviour. `state` variables persist across timesteps, equivalent to registers. They may not instantiate hardware — the bridge between a digital output and a physical primitive is always through a logic port of an analog module or a driven source in a structural module.

Structural modules (plain `module`) are wiring harnesses. They declare signal wires, read MNA node voltages, and instantiate sub-modules. They contain no computation.

10.2 Domain Restrictions

Feature	analog	digital	module
Physical primitives	√	–	√
`V()` / `I()`	√	√	√
Function calls	√	√	√
Math / binary expressions	√	√	–
`if` statement	√	√	–
`while` loop	–	√	–
`state` variables	–	√	–
Standalone assignment	√	√	–
Wire declarations	√	√	√
Signal declarations	√	√	√ *
Sub-module instantiation	√	√	√

* Structural module signal initializers are restricted — see below.

Structural Module Initializer Rules

Signal declarations inside a plain `module` are permitted but their initializer expression is restricted. Only literals, identifiers, and function calls are accepted. Binary arithmetic is not allowed — move computation into a `digital` or `analog` sub-module.

```
// Inside module (structural)
double x = 0;           // allowed -- literal
double x = other_sig;   // allowed -- identifier
double x = V(node);     // allowed -- function call
double x = signal();    // allowed -- function call

double x = -5;          // allowed -- negated literal
double x = a + b;       // ERROR -- binary expression
double x = 2.0 * freq; // ERROR -- binary expression
x = something;          // ERROR -- standalone assignment
```

10.3 Why Two Computational Domains

The separation between `analog` and `digital` reflects a fundamental difference in how each domain interacts with the circuit solver.

Analog modules are solved by the MNA engine. Their outputs — device currents, conductances — are stamped into the matrix and become part of the system $G \cdot x = b$. The solver iterates over analog logic inside its Newton-Raphson loop, recomputing device currents at each iteration until the circuit converges. Analog logic must therefore be purely combinational: given the same node voltages, it must produce the same currents. A `state` variable inside an analog module would accumulate on every Newton-Raphson iteration rather than once per timestep, corrupting the solution.

Digital modules run outside the solver. They execute once per accepted timestep in Phase A,

before the matrix is assembled. Their outputs are treated as fixed inputs to the MNA for that timestep — the solver never re-enters digital logic during Newton-Raphson iteration. This makes `state` variables safe: they update exactly once per timestep regardless of how many inner iterations the solver takes.

In short: analog logic is *inside* the solver loop, digital logic is *outside* it. Mixing the two — a `state` variable in an analog module, or a physical primitive in a digital module — would produce incorrect results, which is why the semantic analyser forbids it.

10.4 Bracket Convention

Brackets	Purpose
`< >`	Static parameters — compile-time constants
`( )`	Logic ports — runtime signals
`[ ]`	Physical ports — electrical wire connections

10.5 Declaration

```
[analog|digital] module Name<type param, ...>(direction type port,
    ...)
     [wire port, ...]
{
     // body
}
```

`direction` is `input` or `output`. Static parameters are evaluated at elaboration time and are available as constants inside the body.

```
// Analog module: device physics, no state
analog module NPN<double Is, double Bf, double Vt>()
     [wire base, wire collector, wire emitter]
{
     double vbe = V(base) - V(emitter);
     if (vbe > 0.75) { vbe = 0.75; }
     if (vbe < -5.0) { vbe = -5.0; }
     double i_be = (Is / Bf) * (2.71828 ** (vbe / Vt) - 1.0);
     current_source<>(i_be)[base, emitter];
}

// Digital module: discrete state, no hardware
digital module PI<>(input double meas, output double ctrl)[]
{
     state double integral = 0;
     double error = 3.0 - meas;
     if (dt > 0) {
          integral = integral + error * dt;
     }
     ctrl = 0.5 * error + 100.0 * integral;
}

// Structural module: wiring only
module main<>()[]
{
     wire motor_p;
     double ctrl_sig = 0;
     double measurement = V(motor_p);
```

```
29
30      module pi_controller = PI<>(measurement, ctrl_sig)[];
31      voltage_source<>(ctrl_sig)[motor_p, gnd];
32      R<1k>()[motor_p, gnd];
33  }
```

10.6 Instantiation

```
module instance_name = ModuleName<static_args>(logic_args)[wire_args
    ];
```

```
module filter = LC_filter<2m, 2u>()[in, lc_out, gnd];
module ctrl   = PI<>(measurement, ctrl_sig)[];
```

`module instance_name =` is mandatory, except for physical primitives.

10.7 Execution Order

Each timestep, Phase A executes logic domains in a fixed causal order:

$$\texttt{module} \longrightarrow \texttt{digital} \longrightarrow \texttt{analog}$$

Structural modules read MNA state first, making voltages and currents available as signals. Digital modules then compute control outputs from those signals. Analog modules run last, computing device currents from the updated node voltages and control signals. Phase B then stamps all driven sources into the MNA matrix before the linear solve.

11. C/C++ Integration (FFI)

Because Hover transpiles circuit descriptions and logic into a high-performance C++ simulation environment, it provides native mechanisms for calling external C or C++ functions directly from your Hover source code. This is achieved using a combination of the `importc` directive and the `extern func` declaration.

11.1 The `importc` Directive

Unlike the standard `import` directive (which evaluates other Hover files), `importc` is a direct pass-through to the C++ code generator. It does not parse the target file. Instead, it instructs the compiler to inject a C++ `#include` statement at the top of the generated simulation code.

```
importc "my_custom_controller.hpp"; // Includes a local header file
importc "<cmath>";                  // Includes a standard C++
    library
```

This ensures that when the final C++ code is compiled, the external function definitions are available to the linker.

11.2 `extern func` Declarations

To call an external C/C++ function, Hover's semantic analyzer must first know its signature (parameters and return type) to enforce type safety during compilation. You declare this using the `extern func` keyword.

An `extern func` declaration acts as a forward declaration. It defines the interface without providing a body:

```
extern func double calculate_gain(double error, double dt, int mode);
extern func int check_status(unsigned int code);
```

11.3 Putting it Together

Once a C++ header is included via `importc` and the function signature is registered via `extern func`, the external function can be called anywhere standard functions are allowed (e.g., inside `digital` or `analog` modules).

```
// 1. Inject the C++ header into the generated output
importc "native_math.h"

// 2. Tell Hover's semantic analyzer about the function signature
extern func double fast_rsqrt(double value);

// 3. Use the external function inside a logic block
digital module RMS_Calculator<>(input double val, output double
    result)[]
{
    state double sum_squares = 0;

    // ... filtering logic ...

    // Call the native C++ function
    result = val * fast_rsqrt(sum_squares);
}
```

11.4 Handling Structs and Complex Types

Because Hover's type system is strictly limited to primitive numeric types (`double`, `float`, `int`, `unsigned`), it does not natively understand C/C++ `structs`, `classes`, or pointers. You cannot directly map a C++ struct as a parameter or return type in an `extern func` signature.

To interface with external C++ code that requires complex types, you must write a **primitive wrapper** on the C++ side.

For example, if you have a native C++ function taking a struct:

```
// C++ struct you want to use
struct MotorParams { double Ld, Lq, Flux; };
double compute_torque(MotorParams p, double id, double iq);
```

You create a "flattened" C++ wrapper function and import that wrapper into Hover:

```
// 1. C++ Wrapper (defined in motor_wrappers.hpp):
// double wrap_compute_torque(double Ld, double Lq, double Flux,
    double id, double iq) {
//     MotorParams p = {Ld, Lq, Flux};
//     return compute_torque(p, id, iq);
// }

// 2. Hover declaration:
importc "motor_wrappers.hpp"
extern func double wrap_compute_torque(double Ld, double Lq, double
    Flux, double id, double iq);
```

11.5 The Opaque Pointer Pattern

With Hover's support for pointers, the most robust way to manage persistent C/C++ structs is by using **opaque pointers**. You can instantiate a struct on the C++ side, cast its memory address to a primitive pointer type (such as `int*`), and pass that pointer back to Hover. Hover simply holds onto the pointer and passes it back to C++ when needed, without needing to know the struct's internal layout.

```
// 1. C++ Wrapper (motor_wrappers.hpp)
struct MotorParams { double Ld, Lq, Flux; };

// Create the struct and return its address as an int*
int* create_motor() {
    MotorParams* p = new MotorParams{0.002, 0.002, 0.15};
    return reinterpret_cast<int*>(p);
}

// Accept the int* from Hover, cast it back, and use it
double compute_torque(int* handle, double id, double iq) {
    MotorParams* p = reinterpret_cast<MotorParams*>(handle);
    return p->Flux * iq; // (simplified for example)
}
```

```
// 2. Hover Script
importc "motor_wrappers.hpp"

extern func int* create_motor();
extern func double compute_torque(int* handle, double id, double iq);

```

```
digital module MotorControl<>(input double id, input double iq,
    output double torque)[]
{
    // Store the opaque pointer as state so it persists across
        timesteps
    state int* motor_handle = create_motor();

    // Pass the pointer back to C++ for computation
    torque = compute_torque(motor_handle, id, iq);
}
```

Warning on Pointer Sizes: Always use true pointer types (like `int*`) to hold memory addresses. Do not attempt to cast or store a pointer inside a regular 32-bit `int` or `unsigned int` variable. Modern operating systems use 64-bit memory addresses; truncating a 64-bit pointer into a 32-bit integer will cause a segmentation fault when passed back to C++.

Note on Compilation: When building a simulation that relies on external FFI files, ensure that the corresponding `.c` or `.cpp` source files are included in your build system (e.g., added to the `Makefile`) so they are correctly compiled and linked with the generated Hover output.

12. Physical Primitives

Physical primitives are stamped directly into the MNA matrix. They use the same bracket convention as modules but do not need a name or `module` keyword.

```
<Primitive> < static_value > ( logic_signal ) [ wire1, wire2 ];
```

Primitive	Parameters	Ports	Description
`R`	`<resistance>`	`[n+, n-]`	Resistor
`L`	`<inductance>`	`[n+, n-]`	Inductor
`C`	`<capacitance>`	`[n+, n-]`	Capacitor
`voltage_source`	`<dc_value>`	`[n+, n-]`	Ideal voltage source
`current_source`	`<dc_value>`	`[n+, n-]`	Ideal current source
`VCCS`	`<gm>`	`[n+, n-, nc+, nc-]`	Voltage-controlled current
`VCVS`	`<k>`	`[n+, n-, nc+, nc-]`	Voltage-controlled voltage
`CCCS`	`<beta>`	`[n+, n-, sense]`	Current-controlled current, sense is a voltage source at 0V
`CCVS`	`<r>`	`[n+, n-, sense]`	Current-controlled voltage, sense is a voltage source at 0V

12.1 Driven Sources (Logic-controlled)

Pass the controlling signal through the logic port `()`:

```
voltage_source<>(generator_output)[in, gnd]; // driven by
    generator_output each step
current_source<>(ctrl_signal)[n+, n-];
```

12.2 Fixed Sources (Compile-time value)

```
voltage_source<12>()[vdd, gnd]; // constant 12 V
R<1k>()[out, gnd];              // 1 kOhm to ground
C<100n>()[node, gnd];           // 100 nF
L<10u>()[a, b];                 // 10 uH
```

13. Built-in Intrinsics

These are available everywhere without import.

Function	Description
`V(wire_name)`	Voltage at MNA node from the previous timestep
`I(element_name)`	not implemented
`idt(x(t))`	integrates the value over time $\int_0^t x(\tau)d\tau$, only works in analog modules
`ddt(x(t))`	derives the value relative to time $dx(t)/dt$, only works in analog modules

```
double vOut  = V(lc_out);   // voltage on net lc_out
double iCoil = I(Vsense);   // current through Vsense branch
double wave  = sin(2.0 * 3.14159265 * 50 * time);
```

Note: `V()` and `I()` always return values from the *previous* MNA solve — they cannot read the result of the current timestep.

13.1 nr_prev()

`nr_prev(expr)` evaluates `expr` using the node voltages and branch currents from the *previous Newton–Raphson iteration* of the current timestep's solve. It is the iteration-level counterpart to `state`: where `state` remembers a value across *timesteps*, `nr_prev()` remembers it across *iterations* within a single timestep's nonlinear solve.

Mechanically, `nr_prev()` does not introduce any storage of its own. It re-emits its argument with every nested `V()` and `I()` call routed to its previous-iteration source (`api_V_prev` / `api_I_prev`) instead of the working-iterate source (`api_V` / `api_I`). Everything else in the expression — parameters, arithmetic, `time`, `dt` — is emitted unchanged. Only `V()` and `I()` carry iteration history, so only they are redirected. Nesting is well defined: `nr_prev()` inside an otherwise ordinary expression, or even `nr_prev()` inside `nr_prev()`, behaves correctly and does not leak the redirection to sibling expressions.

Construct	Value returned
`V(a)`, `I(e)`	most recently solved value (the iterate the solver is currently refining)
`nr_prev(V(a))`	value of `V(a)` one NR iteration earlier, same timestep
`state double x`	value of `x` carried over from the previous timestep

The intended use is writing iteration-aware numerical aids — junction limiting, damping, source ramping — directly in the language rather than hard-coding them into the solver. Because the solver is general (it makes no assumption that a node is a semiconductor junction), the per-device

limiting that keeps a stiff exponential from overflowing belongs in the model. `nr_prev()` supplies the "last guess" that such a limiter needs to bound how far the next guess may move.

```
// Junction limiting expressed in-language: bound the per-iteration
// change in junction voltage using last iteration's value.
double vNew = V(p) - V(n);           // current working iterate
double vOld = nr_prev(V(p) - V(n)); // same junction, previous NR
    iterate

double dv = vNew - vOld;
if (dv >  vt) { dv =  vt; }          // clamp the step, not the
    voltage
if (dv < -vt) { dv = -vt; }
double vLim = vOld + dv;             // limited voltage feeds the
    device model
```

Note: `nr_prev()` only redirects `V()` and `I()`; it is not a general time-domain memory. To remember a value from a previous *timestep*, use `state`. On the first iteration of a timestep's solve there is no earlier iterate, so `nr_prev()` returns that step's starting guess — meaning the very first iteration applies no limiting, which is the intended cold-start behavior.

13.1.1 nr_prev() — Supported Forms

`nr_prev(expr)` accepts *any* expression. There is no special argument grammar: the compiler emits the full translation of `expr` and redirects every nested `V()` and `I()` leaf to its previous-iteration source (`api_V_prev` / `api_I_prev`). Everything else in the expression — parameters, constants, `time`, `dt`, arithmetic, `sin`/`cos` — is emitted unchanged. The redirection is purely lexical over the `V()`/`I()` leaves, so it composes with arbitrary surrounding math.

Form	Emitted (conceptually)	Meaning
`nr_prev(V(a))`	`api_V_prev(a)`	node voltage, previous NR iterate
`nr_prev(I(e))`	`api_I_prev(e)`	branch current, previous NR iterate
`nr_prev(V(a) - V(b))`	`api_V_prev(a) - api_V_prev(b)`	differential voltage, previous iterate
`nr_prev(V(a)) - nr_prev(V(b))`	`api_V_prev(a) - api_V_prev(b)`	identical to the row above
`nr_prev(V(a) / vt)`	`api_V_prev(a) / vt`	`vt` (param) passes through unchanged
`nr_prev(sin(V(a)))`	`sin(api_V_prev(a))`	redirection reaches into call arguments
`nr_prev(nr_prev(V(a)))`	`api_V_prev(a)`	nesting is idempotent (already redirected)

```
double vbe   = nr_prev(V(b) - V(e));        // previous-iterate
    junction voltage
double vbe2  = nr_prev(V(b)) - nr_prev(V(e)); // exactly the same
    code
double iPrev = nr_prev(I(Vsense));          // previous-iterate
    branch current
double scaled= nr_prev(V(a) * 2.0 - V(b));  // params/constants pass
    through
```

Note: The redirection applies *only* to `V()` and `I()`. It does **not** give the previous-iteration value of a computed local or `state` signal — `nr_prev(x)`, where `x` is an ordinary variable, emits `x` unchanged and is therefore a no-op. Likewise `nr_prev(time)`, `nr_prev(dt)`, or any expression containing no `V()`/`I()` leaves reduces to the plain expression. To remember a computed value across iterations, recompute it from `nr_prev`-wrapped `V()`/`I()` reads rather than wrapping the variable.

Note: `nr_prev` reaches into *call arguments* (e.g. `sin(V(a))`), but not into a user function's *body*. If a function internally read `V()` it would not be redirected — though in practice functions cannot take `wire` arguments, so this case does not arise for node reads.

14. Special Keywords

Keyword	Description
`time`	Current simulation time in seconds. Read-only, available everywhere.
`gnd`	Ground node (0 V reference). Available everywhere as a wire.
`state`	Variable modifier — retains value across timesteps.
`dt`	Timestep in seconds. Read-only, available everywhere.
`input`/`output`	Logic port direction in module declarations.

15. Simulation Run Order

Each timestep executes in three phases, repeated until `time` > `stop`.

Phase A **Execute logic — three domain passes in fixed order**

1. ***module*** Structural logic runs first. Wire signals are read from the previous MNA solution via `V()` and `I()` and made available to downstream domains.
2. ***digital*** Digital logic runs second. `state` variables are updated, `if`/`while` blocks are evaluated, and control outputs are written to their signal names.
3. ***analog*** Analog device models run last. Node voltages from the previous solve and control signals from the digital pass are read to compute device currents and physics outputs.

Phase B **Stamp logic outputs into MNA**

Every driven `voltage_source` and `current_source` reads its controlling signal from the values written in Phase A and updates the MNA right-hand side vector b.

Phase C **Solve and advance**

The linear system $G \cdot x = b$ is solved (or iterated to convergence for implicit solvers). Node voltages and branch currents are committed. Signals listed in `.save` are recorded, and `time` advances by `dt`.

Repeat until `time > stop`.

The causal ordering within Phase A is fixed regardless of declaration order in the source file. A digital controller will always see the structural `V()` reads from the current step, and an analog device model will always see the digital control output from the current step.

16. Full Example

Sinusoidal PWM inverter with LC output filter, demonstrating all three module domains working together.

```
.save(main.lc_out, main.generator_output);
.tran(0, 40m, 1u, 10u);
.solver(tr_bdf2);

//-- Helper functions ----------------------------------------

func double sineWave(double freq) {
    double pi = 3.14159265;
    return sin(2.0 * pi * freq * time);
}

func double triangleWave(double freq) {
    double T = 1.0 / freq;
    double t = time;
    while (t > T) {
        t = t - T;
    }
    if (t < T / 2.0) {
        return (4.0 * t / T) - 1.0;
    } else {
        return 3.0 - (4.0 * t / T);
    }
}

//-- Analog module: purely physical, state lives in L and C ----

analog module LC_filter<double inductance, double capacitance>()
    [wire in, wire out, wire ground]
{
    L<inductance>()[in, out];
    C<capacitance>()[out, ground];
}

//-- Digital module: discrete logic, no hardware ---------------

digital module PWM_generator<double freq, double carrier, double vdc>
    (output double target_voltage)[]
{
    double ref = sineWave(freq);
    double tri = triangleWave(carrier);
    if (ref > tri) {
        target_voltage = vdc;
    } else {
        target_voltage = -vdc;
    }
}

```

```
48 //-- Structural module: wiring only ---------------------------
49
50 module main<>()[]
51 {
52     wire in, lc_out;
53
54     // Signal wire: receives PWM output from digital module
55     double generator_output = 0;
56
57     // LC filter: L=2mH, C=2uF -> f0 ~2.5 kHz
58     module filter = LC_filter<2m, 2u>()[in, lc_out, gnd];
59
60     // PWM generator: 50 Hz output, 20 kHz switching, 400 V DC bus
61     module generator = PWM_generator<50, 20k, 400>(generator_output)
           [];
62
63     // Voltage source driven by digital module output each timestep
64     voltage_source<>(generator_output)[in, gnd];
65
66     // 1 kOhm load
67     R<1k>()[lc_out, gnd];
68 }
```

Domain Responsibilities

Domain	Role in this circuit
`analog module`	Stamps L and C into the MNA matrix. The inductor current and capacitor voltage are tracked by the solver automatically.
`digital module`	Compares sine reference against triangle carrier each step and writes ±400 V to `generator_output`.
`module`	Wires the filter and generator together. Routes `generator_output` to the driven voltage source.

Expected Output

Signal	Description
`main.lc_out`	≈ 400 V peak sine at 50 Hz
`main.generator_output`	Square wave ±400 V at 20 kHz

The LC filter passes 50 Hz (below the 2.5 kHz cutoff) and rejects the 20 kHz switching frequency (8× above cutoff, ≈64× attenuation).